

Closing Domains: A Repeatable Method for Finishing Infrastructure

Registry: [\[HOWL-INFO-10-2026\]](#)

Series Path: [\[HOWL-INFO-1-2026\]](#) → [\[HOWL-INFO-2-2026\]](#) → [\[HOWL-INFO-3-2026\]](#) → [\[HOWL-INFO-4-2026\]](#) → [\[HOWL-INFO-5-2026\]](#) → [\[HOWL-INFO-6-2026\]](#) → [\[HOWL-INFO-7-2026\]](#) → [\[HOWL-INFO-8-2026\]](#) → [\[HOWL-INFO-10-2026\]](#)

Date: May 2026

DOI: 10.5281/zenodo.20176608

Domain: Information Theory / Methodology of Production

Domain: Epistemology / Systems Engineering / Cross-Domain Methodology

Status: Methodological Framework with Three Implemented Examples

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

Abstract

Every technical domain has a completable core — a finite set of infrastructure problems that can be built, tested, finished, and closed. This core is routinely obscured by complexity generated by the domain's own tools and preserved as open by the economic and social structure of the community around it. This paper identifies the epistemological prerequisite for recognizing completable cores, defines a four-step method for closing them, and demonstrates the method across three unrelated domains: software execution infrastructure, operational data infrastructure, and physical parameter derivation. The method is repeatable. The prerequisite is the willingness to distinguish falsifiable claims from unfalsifiable ones, commit to what observations support, and build on those commitments as platforms.

1. Information and Contextual Data

Consider two statements.

“Water boils at 100°C at standard atmospheric pressure.” This is a claim about reality. It is independent of who says it. Anyone can test it. It is either true or false, and the test is observable. If someone produces water that does not boil at 100°C under standard atmospheric pressure, the claim is falsified. The claim commits to something, and the commitment can fail visibly.

“Software is inherently complex.” This sounds like a claim about reality. It is phrased declaratively. It appears in conference talks, textbooks, and industry commentary as if it describes a property of software. But it cannot be tested. What observation would show software is not inherently complex? If someone builds simple software, the response is “that’s a simple case” or “real software is complex.” If someone builds complex software, the response is “see, inherently complex.” The statement absorbs all outcomes. It commits to nothing. It cannot fail.

These two statements belong to different categories, and the difference matters structurally.

General information is a falsifiable claim about reality, independent of the observer. It commits to a specific state of affairs. It can be tested. It can be wrong. “The infrastructure layer of software has a finite number of subsystems” is general information — enumerate them and check.

Contextual data is an observer-dependent state. It describes the person holding it, not the world. “I find this architecture elegant” is contextual data. It is real — the person genuinely finds it elegant — but it tells you about the person, not about the architecture. “I prefer chocolate ice cream” is contextual data. It may change tomorrow. It depends on the day, the person’s mood, what they ate for lunch. It is not falsifiable because it does not claim anything about reality external to the observer.

The category error that damages technical fields is treating contextual data as general information. When “software is inherently complex” is treated as information, it enters decision-making as if it were testable. Architectures are chosen to “manage inherent complexity.” Frameworks are designed to “tame inherent complexity.” Careers are built around “addressing inherent complexity.” But the foundation is an unfalsifiable statement — a preference or experience of practitioners, dressed in declarative syntax. The entire structure built on it is unanchored.

This is not a minor philosophical distinction. It has direct engineering consequences. A field that cannot distinguish information from contextual data cannot identify what is settled. If it cannot identify what is settled, it cannot close anything. Every observation gets absorbed into the unfalsifiable framing, and the field remains permanently open — not because the problems are genuinely unsolved, but because the language used to discuss them does not permit the concept of “solved.”

The operating principle of this paper is: every statement is falsifiable. Each claim commits to something specific. Each commitment has a failure condition. Each failure condition is observable. The reader can test each claim independently. If any claim fails, the failure is visible and locates the problem precisely. This is not a rhetorical posture. It is the prerequisite for everything that follows.

2. Commitment and Falsification as Platform

A value has been measured in a well-studied physical system. Every experiment, across decades, measures it as zero. No experiment has ever produced a nonzero result. The theoretical formalism, however, permits the value to range from zero to a maximum. Because the formalism has a slot for nonzero values, the field treats the zero observation as a mystery. “Why is it zero?” becomes a research program. Mechanisms are proposed to explain the zero value. Careers are built around the explanation. Decades pass. The value is still zero. The research program continues.

Consider what has happened structurally. The formalism created a parameter. The observations filled that parameter with zero, consistently. The field, rather than accepting the observation and closing the parameter, treats the formalism’s permission of nonzero values as more authoritative than the observation of zero values. The tool — the theoretical formalism — overrode the measurement.

Two paths were available. The field chose neither.

Path A: Close the parameter. Commit to zero. Build on it as a platform. Every subsequent calculation assumes the value is zero. If anyone ever measures a nonzero value, that is a discovery — clean, dramatic, unmistakable. The commitment was falsified, and something genuinely new is known. Until that falsification, the commitment enables building. Calculations proceed. Results follow. The platform supports weight.

Path B: Demonstrate nonzero. Find an experiment that produces a nonzero value. Publish it. The observation speaks.

What actually happened: Neither path. The parameter is held at “maybe zero.” The field cannot build on zero because it is officially uncertain. The field cannot falsify zero because no one committed to it. The “maybe” absorbs all outcomes. Measure zero again — consistent with maybe, keep looking. Measure nonzero — we always said it could be. No outcome is ever decisive because no position was ever taken.

This pattern — refusing to commit to observed values — has a specific structural consequence: it prevents the use of observations as platforms. A platform is something you build on. Building requires commitment. Commitment requires accepting that the commitment might be wrong. A field that will not accept the possibility of being wrong cannot commit. A field that cannot commit cannot build platforms. A field that cannot build platforms rebuilds its foundations every generation.

The alternative is straightforward. Look at what has been observed. If the observations consistently support a specific value or a specific structural claim, commit to it. State it as a falsifiable position. Build on it. If the commitment holds, the platform supports increasingly ambitious construction. If the commitment fails, the failure is located precisely — the specific claim that was falsified — and the specific point of failure is itself a contribution, because it identifies where reality differs from the commitment.

This is how engineering works. A structural engineer commits to the load-bearing properties of steel. The commitment is based on extensive testing. The engineer builds on it. If a batch of steel fails to meet the committed properties, that is a specific, located

problem. The commitment to steel as a platform is not abandoned — the specific failure is investigated and addressed. The platform holds because the commitment was made.

A field that will not commit to its own observations is a field that cannot engineer.

3. Why Domains Do Not Close

Consider a technical field — any technical field. It has practitioners, institutions, conferences, journals, training programs, tools, and standard practices. It has a history. It has common sense — a set of beliefs that practitioners absorb through participation and transmit to newcomers as background knowledge.

Within this field, there is a set of infrastructure problems. These are problems that every practitioner encounters, that every project must solve, and that have been solved many times before. Text input. Data storage. Network communication. Measurement calibration. Record keeping. The specific problems depend on the field, but the pattern is consistent: a finite set of problems that recur across every project.

These problems are bounded. They have known requirements. They have been solved, often well, many times. Yet they are rebuilt continuously. Each generation of practitioners rebuilds them using the current generation's tools. The rebuilding is not driven by new requirements — the problems have not changed. It is driven by new tools.

The mechanism is tool-generated complexity. Every tool embodies a formalism. Every formalism creates a space of possibilities. Some of those possibilities correspond to genuine variability in the world — real problems that need real solutions. Some are artifacts of the formalism — questions that only exist because the tool created a slot for them.

Consider a programming language that requires every data transformation to be expressed as an object with methods. The language creates the question “what class hierarchy should represent this transformation?” The question is real within the language. It is an artifact of the language's formalism. A different tool — one that expresses data transformations as data flowing through stages — does not generate that question. The problem was never in the transformation. It was in the tool.

This compounds across a field. Each tool generates its own questions. Each question generates work. Each unit of work generates expertise. Each expert has an incentive to preserve the tool that generated their expertise. The field's common sense accumulates these tool-generated questions as “the fundamental challenges of the domain.” Newcomers learn the tools, encounter the questions, and accept them as inherent.

The completable core — the finite set of infrastructure problems — is obscured. Not hidden, exactly. Any practitioner can enumerate the problems if asked directly. But the tool-generated complexity creates a forest around each problem that makes the problem appear larger and more open than it is. “Text input” becomes “the challenge of cross-platform input handling across framework paradigms with accessibility compliance and internationalization support integrated into the reactive state management pipeline.” The

problem did not grow. The description of the problem grew, because each tool added its own vocabulary.

Reference values create an additional obstacle in fields that depend on measurement. A reference value is a number published by a committee as the official current best estimate of some quantity. The committee takes readings from multiple instruments, each of which gives a different value each time. The readings are averaged, weighted, adjusted, and published. The published value changes each cycle — not because the underlying quantity changed, but because the committee’s methodology changed.

Results in the field are evaluated against these reference values. A derived result that disagrees with the reference by 0.1% is reported as “0.1% discrepancy.” But the reference itself shifted by a comparable amount since the last publication cycle. The derived result may be more stable than the reference. This cannot be determined within the framework of comparing against committee outputs, because the committee output is treated as ground truth by convention, not by any structural argument about its accuracy.

The combined effect is a field that cannot recognize closure. The tools generate complexity that makes bounded problems appear open. The reference values shift, making stable results appear imprecise. The common sense of the field — “our domain is inherently ongoing” — is unfalsifiable contextual data that gets transmitted as information. Each generation inherits the tools, the reference values, and the common sense, and rebuilds the completable core.

This is not conspiracy. It is ordinary human behavior under ordinary incentives. The practitioners are skilled, often brilliant, and working in good faith. They are also embedded in an economic and social structure that depends on the domain remaining open. The structure doesn’t need to coordinate to preserve itself. It preserves itself through the aggregated individual incentives of its participants, each of whom has good local reasons for the choices they make.

4. The Method

The method has four steps. Each step is defined here in general terms. The following three sections demonstrate each step concretely in three different domains.

Step 1: Take the key results of the best research seriously.

Every field has landmark results. These are findings that the community acknowledges, cites, and admires. They appear in textbooks. They are mentioned in keynotes. They are treated as interesting contributions to the field’s understanding.

They are not followed to their structural conclusions.

The reason is usually that the structural conclusion threatens the field’s current practice. The result is acknowledged in a way that contains it — “an important perspective” — and then practice continues unchanged. The result is true and the practice contradicts it, and

both states coexist because the community treats the result as a contribution to discussion rather than a constraint on action.

Identify these results. They are the key to the completable core. They typically identify a structural property of the domain that, taken seriously, simplifies everything built on top of it. They are usually old. They are usually well-tested. They are usually considered settled within their narrow scope but not generalized.

Take them as literally true. Not as interesting perspectives. Not as useful sometimes. As structural properties of the domain that constrain what valid infrastructure looks like.

Step 2: Unify the language.

The field's standard vocabulary was built around its tools. Different tools generated different vocabularies. The vocabulary carries the tool-generated complexity. Translating between vocabularies is itself treated as a research problem.

Replace the vocabulary with language that is consistent with the key results from Step 1 and that is falsifiable. This typically means discarding distinctions that only exist because different tools created them and introducing distinctions that correspond to observable structural properties.

The unified language will be simpler than the field's standard language. If it is not simpler, the key results have not been taken seriously enough — some tool-generated complexity has been preserved in the language.

Step 3: Implement as substrate.

Build the infrastructure that embodies the unified language. This is engineering — evaluating trade-offs against a substrate that pushes back, where failure has real consequences. The implementation must match the language, and the language must match the key results. Every design decision is a commitment. Every commitment is falsifiable.

The substrate is the platform. Building it is the act of commitment described in Section 2 — the claim that this infrastructure is sufficient and closeable. The claim can be wrong. If it is wrong, the failure will be visible at a specific point.

Step 4: Test across domains within the field.

Run the substrate against multiple application areas within the field. Each application exercises the substrate differently. Each application is a test of the commitment from Step 3.

Three outcomes are possible. The substrate handles the application without extension — the commitment holds for that application. The substrate requires extension — the commitment was incomplete at a specific point, and the extension rate is measurable. The substrate cannot handle the application at all — the commitment is falsified at a specific point, and the failure identifies where the substrate's design is wrong.

The convergence signal is the extension rate. Early applications may require frequent extensions because the substrate is young. Later applications require fewer. If the rate approaches zero — new applications stop requiring new infrastructure — the substrate is

approaching closure. If the rate does not decrease, the substrate is not converging, and either the design is wrong or the domain is genuinely not completable. Both are useful results. Both are located precisely.

5. Application: Software Execution Infrastructure

Step 1: The key result. The CPU is a data transformation machine. It does not know what an object is, what a class hierarchy is, what a design pattern is. It moves data from one location to another, transforms it, and stores the result. Every program, from trivial to complex, is a data transformation pipeline: data enters, flows through stages, gets transformed, exits. This observation was made forcefully in the context of high-performance software design, where aligning data layout with the CPU's actual processing model — cache-friendly structures, batched operations, no pointer chasing — produced dramatic performance improvements. But the observation extends beyond performance. It is architectural. Every abstraction in software — objects, services, frameworks — is an attempt to organize data transformation. The abstraction describes the code that performs the transformation, not the transformation itself.

Taking this seriously means: if all software is data transformation, then the infrastructure that performs data transformation is a finite engineering problem. The transformations are enumerable. The pipeline stages are enumerable. The compiled binary that implements the pipeline has a bounded scope.

Step 2: Language unified. Traditional software fuses two things into a single binary: infrastructure (the pipeline that transforms data) and application logic (the specific transformations to perform). A web server parses requests, routes them, processes them, constructs responses, and sends them — that is infrastructure. The specific routes, the specific processing, the specific responses — that is application logic. A game renders frames, processes input, evaluates AI, applies physics — that is infrastructure. The specific enemies, the specific behaviors, the specific levels — that is application logic.

The unified language separates these absolutely. Infrastructure is the compiled binary. It processes arbitrary data transformations. It contains zero domain types — no knowledge of what the data means. Application logic is a dataset that the binary interprets. It defines what entities exist, what states they can be in, what rules govern transitions, what behaviors are available, and what effects actions produce. The dataset teaches the binary what to be. The binary provides the mechanics. The dataset provides the semantics.

This is distinct from “data-driven development,” which has been practiced for decades. Data-driven means the programmer writes a class with compiled behavior and the designer adjusts numeric parameters. The behavior is still compiled. The distinction is: does the compiled binary know any domain noun exists? If the source code contains “goblin” or “sword” or “quest” as types, enums, or string comparisons that drive behavior, compiled behavior is present. Data-only means the binary has never heard of these things. They exist only in the dataset.

Step 3: Substrate implemented. The execution pipeline is a funnel. Each layer narrows the candidate set.

The first layer is a state machine — pure topology. Named states with transitions between them. A state contains no behavior. It references a behavior set and defines exit conditions: events, durations, and logical rules evaluated each frame. The state machine determines which behaviors are available.

The second layer is predicate logic. Rules are evaluated against facts regenerated every frame from entity state. Does the entity have a target? What is the distance? Is health below a threshold? The evaluator performs pattern matching. It does not know what “target” or “health” mean. It matches predicates against facts.

The third layer is utility scoring. Available behaviors are scored multiplicatively. Each behavior has considerations — inputs normalized to a zero-to-one range, shaped by curves, weighted and multiplied. Any consideration that evaluates to zero eliminates the behavior entirely. The highest-scoring behavior wins. Behaviors with more considerations naturally score lower but are more specific. Specificity is self-balancing through the mathematics.

The fourth layer is a bytecode interpreter with approximately one hundred block types for control flow, arithmetic, logic, and data access. The interpreter reads and writes entity fields via paths resolved at runtime. It cannot crash: invalid paths return default values, arithmetic operations clamp and saturate, array access is bounds-checked.

The fifth layer is envelope processing — time-bounded modifications to values, shaped by curves. A modification with a duration, a tick rate, a target value, a modifier amount, and a curve shape. This is the core: values modified over time. A combat strike, a healing effect, a financial transaction, a temperature change — all parameterizations of the same operation. The infrastructure applies the modifier to the target, shaped by the curve, over the duration. The difference between a combat strike and a financial transaction is not code. It is data: different target, different modifier, different duration, same pipeline.

Every entity has the same structure. Fields labeled “health” on one entity can be relabeled “revenue” on another through a replacement table. The data underneath is identical — the same numeric value processed by the same pipeline. Only the label changes. The pipeline does not care what the label says.

A scene is an isolated execution context with its own entity pool, its own time tracking, its own permissions. Multiple scenes run simultaneously — a game, a data browser, a debugger, a performance monitor — each isolated, each managed by a scene scheduler. The tools that inspect the running application are themselves scenes, built on the same infrastructure they inspect.

Step 4: Tested across domains. Games of multiple genres. Data browsers. Debugging tools. Performance monitors. Level editors. Each application is a dataset interpreted by the same binary. Each application exercised the pipeline differently. The extension rate — the rate at which new applications required new infrastructure primitives — decreased with each application. The pipeline approached closure.

The binary contains zero domain types. It does not know whether it is running a game,

a spreadsheet, a chat client, or a business dashboard. All of these are datasets that teach the pipeline what to be.

Falsification condition: if an application domain cannot be expressed as datasets on the pipeline without adding compiled domain logic to the binary, the thesis fails at that point.

6. Application: Operational Data Infrastructure

Step 1: The key result. Data outlives logic. Goals change. When goals change, the logic that served those goals becomes obsolete. The data that the logic operated on persists — it describes reality, not intent. Operational tools churn every decade. The operational data they managed — what servers exist, who owns them, what certificates expire when, what the on-call rotation is — persists across every tool transition. The data is the durable layer. The tooling is the disposable layer.

A second key result: comprehensive systems are built top-down from the whole. Aggregated systems are built bottom-up, piece by piece, and trend toward inconsistency. A system built by assembling parts never becomes whole — it reaches a ceiling where adding more parts breaks previously added parts. A system built by subdividing a whole maintains internal consistency because every piece was cut from the same cloth.

Taking these seriously means: operational infrastructure should be organized as a data substrate with a comprehensive schema, and the tooling that acts on that data should be separate, replaceable, and disposable.

Step 2: Language unified. An operational environment is a distributed system managed as a coordinated whole. Within this environment, there are authorities — systems that own specific slices of operational reality. A container orchestrator owns workload state. A secrets manager owns credentials. A monitoring system owns metrics. No single system owns all of operational reality.

The unified language introduces a centralized passive data substrate: a single source of truth for the coordination layer. It stores configuration, schedules, policies, runner metadata, documentation metadata, authority pointers, and change management state. It does not store what the authorities own — it stores pointers to where that data lives. It does not invoke work — it answers queries and accepts writes. Runners — small single-purpose automations — consult the substrate, perform work via shared libraries, and write results back.

The substrate is passive. This commitment is structural, not stylistic. If the substrate invokes work, it becomes an orchestrator — stateful, complex, fragile, and entangled with the logic it was supposed to outlive. Passivity is the boundary that keeps the data layer durable.

Step 3: Substrate implemented. The schema covers operational reality: centrally-managed configuration, cached observed state from authorities, typed pointers to where facts live, a service directory for operational questions, schedules and calendars, runner

deployment metadata, documentation metadata with ownership and review dates, declarative policies for security and compliance and retention, version history per entity, an append-only audit log, change management state, and cross-substrate references for environments with multiple instances.

The API enforces all governance uniformly. Authentication via identity provider integration. Authorization by group and role, scoped to entity type, namespace, and field. Validation against schema, value constraints, referential integrity, and semantic checks. Change management with proposal, routing, approval, and atomic commit. Automatic versioning on every committed change. Append-only audit of every action. Rate limiting per identity and endpoint.

The substrate is storage-engine independent. The schema is the long-lived artifact. The engine underneath can be replaced without changing the schema, the API contract, or the runners. The schema outlives the storage engine, the API implementation, the runner implementations, and the shared library implementations. This is by design — each layer has a different churn rate. The schema changes slowly. The shared libraries change at medium pace. The runners change quickly. The integrations with external systems change fastest.

Structural commitments prevent drift. The substrate never invokes work — preventing orchestrator drift. The API is the only path — no out-of-band access, no shadow paths, no exceptions except audited maintenance. The cardinality is one or many, never two — two instances create split-brain coordination, drifting schemas, and uncoordinated governance, which is a failure state, not a valid architecture. New use cases are absorbed into the existing substrate rather than served by additional instances.

Step 4: Tested across domains. Network operations. Server lifecycle management. Security policy enforcement. Compliance evidence collection. Deployment coordination. Certificate management. Credential rotation. Backup verification. Vendor contract tracking. On-call scheduling. Each operational domain was examined for fit against the schema. Each domain was absorbed through new entity types, relationships, or namespaces — not through schema redesign. Each runner built against the substrate confirmed that the schema was useful, not merely elegant.

The convergence signal: each new domain refined the top-level taxonomy without replacing it. The schema stabilized. New domains required less adaptation than previous ones. The substrate approached closure.

Falsification condition: if an operational domain cannot be absorbed into the entity model without violating the schema's structural commitments, the thesis fails at that point.

7. Application: Physical Parameter Derivation

Step 1: The key result. Quantum electrodynamics produces the most accurate predictions in the history of science. It works by counting. Discrete particles, discrete interactions, discrete channels. You count the particles, count the channels, perform the arith-

metic, and the result agrees with observation to twelve decimal places. No other theory in any field achieves this precision.

The standard model of particle physics has approximately twenty-five free parameters — coupling constants, particle masses, mixing angles. These values are not derived from the theory. They are measured by experiments, averaged by committees, and inserted into the equations by hand. The equations then produce predictions. The predictions agree with further measurements. But the parameters themselves remain unexplained — slots in the formalism filled from outside.

Taking the QED result seriously means: the most accurate theory in science uses integers. Counting works. The standard model equations work. What if the free parameters are not free — what if they can be derived from integers using the same equations, if the number representation is changed?

Step 2: Language unified. The standard model equations are expressed using continuous real numbers. Transcendental numbers — π , e , natural logarithms — appear throughout. Elliptical integrals appear in higher-order corrections. In practice, these are implemented as floating-point arithmetic — typically sixty-four-bit IEEE 754. Floating-point arithmetic approximates real numbers. The approximation introduces rounding. The rounding accumulates over long derivation chains. The accumulated error is absorbed into error bars reported on final results.

The unified language replaces continuous reals with exact rationals. Every transcendental and elliptical integral is approximated as a rational number with a fixed common denominator — a power of two large enough that the approximation exceeds the precision of any instrument. The equations do not change. The physics does not change. The computational substrate changes from approximate floating-point to exact rational arithmetic.

Exact rational arithmetic does not round. It does not accumulate error. Intermediate values are exact to the last digit. Long derivation chains are stable — the hundredth step is as precise as the first. The tool contributes zero uncertainty to the result. Any disagreement between the derivation's output and an observation is in the physics or in the measurement, not in the computation.

Step 3: Substrate implemented. The standard model textbook equations, unchanged, run with exact rational arithmetic. The common denominator is two to the three hundred and thirty-fifth power — far more precision than any instrument can test, and a power of two so that the arithmetic is binary-native.

The execution infrastructure is a data pipeline. Experiments are defined as structured data files. Each file specifies named value inputs, named derivation steps, and named comparisons with explicit pass/fail criteria. A runner loads the experiment, loads the value store, executes the derivations in order, performs the comparisons, and writes versioned results. The runner does not know what physics is. It processes named values through named transformations and checks named conditions.

The value store contains approximately five thousand named values — physical constants, measured reference values, intermediate derivation results, integer inputs. Values are

named, typed, versioned, and searchable. The store is the single source of truth for the computation. Every derivation draws from it and writes back to it.

Step 4: Tested across domains within physics. Fifty experiments spanning the major domains of physics: quantum electrodynamics, electroweak theory, Big Bang Nucleosynthesis, cosmology, dark matter, general relativistic time dilation, quark mixing, the muon anomalous magnetic moment, the Hubble expansion rate, and the confinement boundary of the strong force.

The results are cross-domain. The integers that produce the electromagnetic coupling constant in QED produce the baryon density parameter in cosmology. The baryon density produces the dark-matter-to-baryon ratio in dark matter physics. One value store, one derivation chain, domain boundaries crossed by computation rather than by inserting independently measured values. If the QED integers are wrong, the cosmology results break. If the cosmology derivation is wrong, the dark matter predictions break. The chain is falsifiable end to end.

Selected results against reference values. Baryon density parameter: derived 0.049036, measured 0.049, miss 0.073%. Dark-matter-to-baryon ratio: derived 5.317, measured 5.320, miss 0.073%. Helium-3 abundance: derived within one standard deviation of measurement. Tully-Fisher velocity scaling: exactly sixteen, an exact integer result from exact arithmetic. Amplification factor in dark matter: forty-four thirteenths, an exact rational derived from the same integer framework.

The lithium-7 prediction is instructive. Standard Big Bang Nucleosynthesis predicts a lithium-7 abundance roughly three times higher than what is observed in old stars. This tension — the “lithium problem” — has been unresolved for decades. The integer framework reproduces the same tension: predicted 4.74×10^{-10} , observed 1.6×10^{-10} , ratio approximately three. This confirms that the tension is in the physics, not in the parameterization. When the free parameters are derived from integers rather than measured and inserted, the same discrepancy appears. Whatever is happening with lithium-7 is real.

The reference values these results are compared against are themselves committee averages of noisy instrument readings that change each publication cycle. The integer derivations do not change. Run the computation today, run it in ten years, same output. The reference values will differ in ten years — not because the universe changed but because the committee reprocessed the readings. The integers are at least as stable as the universe. The reference values are less stable than either.

Falsification condition: if a measurement disagrees with the integer derivation beyond the measurement’s own stated uncertainty, the derivation fails at that point. Each experiment’s comparisons make this explicit — pass, fail, or informational, with miss percentages and sigma values.

8. The Pattern

Three domains. Three applications of the same four-step method. The structural features they share are consequences of the method, not coincidences.

A completable core was identified and separated from an ongoing layer. In software: infrastructure separated from applications. In operations: schema separated from runners. In physics: derived parameters separated from experimental measurements. In each case, one layer converged and approached closure. The other layer remained open — and correctly so, because it tracks human goals and empirical observations, which change.

Tool-generated complexity was eliminated. In software: class hierarchies, framework churn, the tension between adding new types and new operations — artifacts of programming languages that fuse data transformation with code organization. The data-only pipeline does not generate these questions because it does not have the formalism slots that created them. In operations: operational tool fragmentation, competing coordination platforms, migration projects between tooling generations — artifacts of fusing the coordination substrate with the work execution layer. The passive data substrate does not generate these questions. In physics: free parameters, domain separation, the “mystery” of why certain values are what they are — artifacts of a continuous formalism implemented in approximate floating-point arithmetic. Exact rational arithmetic over a fixed integer lattice does not generate these questions.

In each case, the eliminated complexity was real work that real practitioners performed. The work was skilled, sometimes brilliant. It was also unnecessary — generated by the tools rather than by the problems. The problems remained after the tool-generated complexity was removed. They were smaller than they appeared. They were finite. They were closeable.

Cross-domain testing revealed convergence. In software: new applications stopped requiring new infrastructure primitives as the pipeline matured. In operations: new operational domains stopped requiring schema redesign as the taxonomy stabilized. In physics: integer derivations from one domain produced correct results in another — QED integers predicted cosmological parameters, cosmological parameters predicted dark matter ratios. Convergence is the observable signal that a substrate is approaching closure.

Falsification conditions are explicit throughout. Every major claim has a stated failure mode. Every failure mode is observable. This is not incidental. It is the method itself — the method only works because each commitment can fail visibly. A commitment that cannot fail cannot serve as a platform, because there is no way to determine whether the platform is holding.

9. Applying the Method

The method does not depend on the three domains in which it was demonstrated. It depends on properties common to any domain that has a completable core. The following is a practical process for applying it.

Identify the key results. What has the field established that the community acknowledges but does not build on? These results are typically old, well-tested, and broadly accepted within their narrow scope. They are treated as contributions to understanding rather than as constraints on practice. Look for results that, if taken literally, would simplify the field's infrastructure. Look for results that practitioners cite in talks but contradict in daily work. The gap between citation and practice marks the key result.

Separate information from contextual data. Examine the field's common-sense beliefs. For each belief, ask: what observation would falsify this? If the answer is "none" — if the belief absorbs all outcomes — it is contextual data, not information. It describes the community, not the domain. It may be widely held, it may feel true, it may be spoken by respected figures. It is still not information if it cannot be falsified. Set it aside. What remains after removing the unfalsifiable beliefs is the set of actual structural claims the field has established. These claims are the candidates for platform commitments.

Identify tool-generated complexity. For each problem the field considers open, ask: does this problem exist because of the domain or because of the tools used to work in the domain? Would this problem exist if the tool were different? If the problem disappears under a different tool, it was generated by the tool. It is real work — practitioners genuinely spend time on it — but it is not a domain problem. It is a tool problem. The domain problem it obscures is smaller.

Unify the language. Express the domain in terms consistent with the key results. Use language that is falsifiable. Discard vocabulary that only exists because different tools created different terminology for the same operations. The unified language should be simpler than the field's standard vocabulary. If it is not simpler, tool-generated complexity remains in the language. Simplify further.

Commit and build. Close the parameters that observations support closing. Build the substrate that embodies the unified language. This is the commitment — the falsifiable claim that this platform works. Every design decision in the substrate is a specific commitment with a specific failure mode. Document the failure modes. They are the most important part of the design, because they are what make the design testable.

Test across domains within the field. Run the substrate against multiple application areas. Measure the extension rate — how often new applications require new substrate primitives. If the rate decreases, the substrate is converging. If it reaches zero, the substrate is approaching closure. If it does not decrease, either the substrate design is wrong at specific identifiable points, or the domain is genuinely not completable — and both of these are useful, located results.

Expect social resistance. The community's structure depends on the domain remaining open. The observation that the domain can close threatens livelihoods, identities, and institutions. This resistance will manifest as objections. Evaluate each objection by asking whether it offers a falsifiable counter-claim or an unfalsifiable objection. A falsifiable counter-claim is valuable — it identifies a specific point where the substrate might be incomplete, and testing it either strengthens the substrate or improves it. An unfalsifiable objection — "that could never work for real systems" or "you don't understand the inherent

complexity” — is contextual data about the objector. It tells you about their experience and their relationship to the field. It tells you nothing about the substrate.

Not all objections will be unfalsifiable. Some will identify genuine gaps. This is the method working correctly. The substrate improves through falsification of specific commitments. Every identified gap makes the substrate better. The goal is not to be right about every commitment on the first attempt. The goal is to make every commitment testable so that failures are located and addressed, and successes accumulate into a platform.

10. Closing

Every technical domain has a completable core. The core consists of infrastructure problems that are finite, enumerable, buildable, testable, and closeable. The core is obscured by complexity generated by the domain’s own tools and preserved as open by the economic and social incentives of the community built around it.

The method for closing the core is: take the field’s strongest results seriously as structural constraints, unify the language to be consistent with those results and falsifiable, build the substrate that embodies the unified language, and test it across domains within the field. The extension rate either converges toward zero — approaching closure — or it does not — identifying specific points of failure.

The prerequisite is epistemological. Separate general information — falsifiable claims about reality — from contextual data — observer-dependent states that describe the observer, not the world. Commit to what the observations support. Build on the commitment. Accept falsification when it comes. This is the operating principle that makes engineering possible in any domain.

Three domains were closed using this method. Software execution infrastructure: a compiled binary containing zero domain types, processing arbitrary datasets through a fixed pipeline, approaching closure as new applications stop requiring new primitives. Operational data infrastructure: a passive data substrate with a comprehensive schema, absorbing new operational domains without redesign, with the schema stabilizing as each domain is added. Physical parameter derivation: standard model equations run with exact rational arithmetic, deriving previously free parameters from integers, with cross-domain derivation chains holding across QED, cosmology, and dark matter.

Each closed substrate became a platform. Platforms enable building. The creative, goal-driven, ever-changing layer runs on top: applications on the software binary, runners on the operational schema, experiments on the integer framework. The ongoing layer is correctly ongoing — it tracks human goals and empirical observations, which change. The infrastructure layer underneath is correctly finished — it processes what it is given, without knowing what it means.

The method is repeatable because it depends on properties common to all domains with completable cores: the existence of underused key results, the distinguishability of tool-

generated from genuine complexity, the possibility of unified falsifiable language, and the testability of cross-domain convergence. It does not depend on the specific domain. It depends on the willingness to commit.

References

- [1] M. Acton, “Data-Oriented Design and C++,” CppCon 2014.
- [2] K. Popper, *The Logic of Scientific Discovery*, 1959.
- [3] Planck Collaboration, “Planck 2018 results. VI. Cosmological parameters,” *Astronomy & Astrophysics*, vol. 641, A6, 2020.
- [4] Particle Data Group, “Review of Particle Physics,” *Progress of Theoretical and Experimental Physics*, 2022.
- [5] S. Weinberg, “The cosmological constant problem,” *Reviews of Modern Physics*, vol. 61, no. 1, pp. 1–23, 1989.
- [6] L. Sbordone et al., “The metal-poor end of the Spite plateau,” *Astronomy & Astrophysics*, vol. 522, A26, 2010.
- [7] R. D. Peccei and H. R. Quinn, “CP Conservation in the Presence of Pseudoparticles,” *Physical Review Letters*, vol. 38, no. 25, pp. 1440–1443, 1977.
- [8] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” *USENIX ATC*, 2014.
- [[HOWL-INFO-10-2026](#)] Closing Domains: A Repeatable Method for Finishing Infrastructure. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-10-2026>
- [[HOWL-INFO-1-2026](#)] Multi-Dimensional Information Indexing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-1-2026>
- [[HOWL-INFO-2-2026](#)] The Scales Method. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-2-2026>
- [[HOWL-INFO-3-2026](#)] The Pseudo-Socratic Method. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-3-2026>
- [[HOWL-INFO-4-2026](#)] Howland’s Axiom of Information Locality. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-4-2026>
- [[HOWL-INFO-5-2026](#)] Coherence as Information Phenomenon. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-5-2026>

[[HOWL-INFO-6-2026](#)] How to Write Technical Documents. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-6-2026>

[[HOWL-INFO-7-2026](#)] How to Review Technical Writing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-7-2026>

[[HOWL-INFO-8-2026](#)] LLMs are not Tools, LLMs are Maybe-Tools. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-8-2026>